

UniApp API 交互反馈：showToast 用法全解析

在 UniApp 跨平台开发中，`uni.showToast()` 是最常用的交互反馈 API 之一，用于在页面中间显示轻量级提示信息，告知用户操作结果（如“操作成功”“参数错误”）或当前状态（如“加载中”）。其核心优势是多端自适应（自动适配微信小程序、App、H5 等平台样式），且使用简单、配置灵活。本文将从基础用法、参数详解、场景实战到避坑指南，完整覆盖 showToast 的核心知识点。

一、核心认知：showToast 的基础特性

在使用前需明确 showToast 的核心特性，避免与其他提示类 API（如 showModal、showLoading）混淆：

- 显示位置**：固定在页面垂直居中、水平居中位置，无法自定义位置（如需自定义位置，需使用自定义组件）。
- 自动关闭**：默认 1.5 秒后自动关闭，也可通过 `duration` 参数控制关闭时间，无需手动调用关闭方法。
- 图标限制**：支持官方预设图标（成功、错误、加载、无图标），也可自定义图片图标，但图片尺寸和格式有明确要求。
- 层级关系**：提示框层级较高，会覆盖页面大部分内容，但不阻断页面事件（部分平台在显示期间会短暂禁止点击）。
- 多端差异**：各平台样式略有不同（如 App 端提示框背景更圆润，H5 端与浏览器样式融合），但核心功能一致。

二、基础用法：语法与最简示例

showToast 采用回调式语法（支持 Promise 链式调用），基础配置仅需指定提示内容（`title`），即可快速实现提示功能。

1. 语法结构

Code block

```
1  // 回调函数形式
2  uni.showToast({
3    title: '提示内容', // 必选参数
4    icon: 'success', // 可选，图标类型
5    duration: 2000, // 可选，显示时长
6    success: (res) => { // 可选，调用成功回调
7      console.log('提示显示成功', res)
```

```

8      },
9      fail: (err) => { // 可选, 调用失败回调
10         console.error('提示显示失败', err)
11      },
12      complete: () => { // 可选, 调用完成回调 (成功/失败都执行)
13         console.log('提示调用完成')
14      }
15 })
16
17 // Promise 形式 (UniApp 支持, 更适合异步流程)
18 try {
19     const res = await uni.showToast({
20         title: 'Promise 提示',
21         icon: 'none'
22     })
23     console.log('显示成功', res)
24 } catch (err) {
25     console.error('显示失败', err)
26 }

```

2. 最简示例：基础文本提示

仅指定 `title` 时，默认显示“无图标”样式，1.5 秒后自动关闭，适用于简单的文本提示场景（如“请输入用户名”）：

Code block

```

1  <template>
2    <button @click="showBasicToast">显示基础提示</button>
3  </template>
4
5  <script setup>
6    const showBasicToast = () => {
7      uni.showToast({
8        title: '请输入完整信息' // 仅必选参数
9      })
10   }
11 </script>

```

三、核心参数详解：定制提示样式与行为

`showToast` 的参数丰富，可通过配置实现不同图标、时长、样式的提示，核心参数如下表所示，其中 `title` 为唯一必选参数。

1. 核心参数表

参数名	类型	可选值/说明	默认值
title	String	提示文本，长度建议不超过15字	无
icon	String	success/error/loading/none	none
duration	Number	提示显示时长（毫秒），范围1000-10000	1500
mask	Boolean	true/false	false
image	String	自定义图标的本地路径（不支持网络图片）	无
position	String	top/middle/bottom（仅App端支持）	middle

2. 关键参数实战示例

2.1 带官方图标的提示（常用场景）

通过 `icon` 参数指定官方图标，适用于明确的操作结果反馈（如成功、错误）或状态提示（如加载中）：

Code block

```
1  <script setup>
2  // 1. 操作成功提示 (success 图标)
3  const showSuccessToast = () => {
4    uni.showToast({
5      title: '提交成功',
6      icon: 'success', // 绿色对勾图标
7      duration: 2000 // 显示2秒
8    })
9  }
10
11 // 2. 操作错误提示 (error 图标)
12 const showErrorToast = () => {
13   uni.showToast({
14     title: '提交失败，请重试',
```

```

15     icon: 'error', // 红色叉号图标
16     mask: true // 显示遮罩，防止点击穿透
17   })
18 }
19
20 // 3. 加载中提示 (loading 图标，需手动关闭)
21 const showLoadingToast = () => {
22   uni.showToast({
23     title: '加载中...',
24     icon: 'loading', // 旋转加载图标
25     mask: true,
26     duration: 10000 // 延长时长，避免提前关闭
27   })
28
29   // 模拟接口请求，完成后关闭提示
30   setTimeout(() => {
31     uni.hideToast() // 手动关闭提示
32   }, 3000)
33 }
34 </script>

```



注意： `icon: 'loading'` 时，默认不会自动关闭，需在操作完成后调用 `uni.hideToast()` 手动关闭，否则提示会一直显示。

2.2 自定义图标提示

当官方图标无法满足需求时，可通过 `image` 参数设置自定义本地图标，图标建议为 40x40px 的 PNG 格式，优先级高于 `icon` 参数：

Code block

```

1  <script setup>
2  const showCustomIconToast = () => {
3    uni.showToast({
4      title: '积分+10',
5      image: '/static/icons/score.png', // 本地图标路径 (static目录下)
6      duration: 1500
7    })
8  }
9  </script>

```

注意：`image` 仅支持本地路径，不支持网络图片；若同时设置 `icon` 和 `image`，`image` 会覆盖 `icon` 生效。

2.3 带遮罩的提示

设置 `mask: true` 可显示透明遮罩，防止用户在提示显示期间点击页面其他元素，适用于重要操作反馈或加载状态：

Code block

```
1 uni.showToast({
2   title: '处理中，请勿操作',
3   icon: 'loading',
4   mask: true // 遮罩生效，页面不可点击
5 })
```

四、常见使用场景与最佳实践

`showToast` 适用于各类轻量级交互反馈，结合不同场景优化配置，可提升用户体验。

1. 表单验证反馈

用户提交表单时，若验证不通过，使用无图标提示告知具体错误，提示文本需清晰明确（避免模糊表述）：

Code block

```
1 <template>
2   <view class="form">
3     <input v-model="username" placeholder="请输入用户名" />
4     <input v-model="password" type="password" placeholder="请输入密码" />
5     <button @click="submitForm">登录</button>
6   </view>
7 </template>
8
9 <script setup>
10  import { ref } from 'vue'
11
12  const username = ref('')
13  const password = ref('')
14
15  const submitForm = () => {
16    // 表单验证
17    if (!username.value) {
18      uni.showToast({ title: '请输入用户名' })
19    }
16    return
```

```
20     }
21     if (password.value.length < 6) {
22         uni.showToast({ title: '密码需至少6位' })
23         return
24     }
25
26     // 验证通过，执行登录逻辑
27     uni.showToast({ title: '登录中...', icon: 'loading', mask: true })
28     // 后续接口请求...
29 }
30 </script>
```

2. 接口请求反馈

接口请求前显示加载提示，请求成功/失败后显示对应结果提示，形成完整的交互闭环：

Code block

```
1 // 模拟获取用户信息接口
2 const getUserInfo = async () => {
3     try {
4         // 1. 请求前显示加载提示
5         uni.showToast({ title: '加载中...', icon: 'loading', mask: true, duration:
10000 })
6
7         // 2. 发起接口请求
8         const res = await uni.request({
9             url: 'https://api.example.com/user',
10            method: 'GET'
11        })
12
13        // 3. 请求成功，关闭加载提示并显示结果
14        uni.hideToast()
15        uni.showToast({ title: '加载成功', icon: 'success' })
16
17        // 处理接口数据...
18    } catch (err) {
19        // 4. 请求失败，关闭加载提示并显示错误
20        uni.hideToast()
21        uni.showToast({ title: '加载失败，请重试', icon: 'error' })
22    }
23 }
```

3. 操作结果反馈

用户完成点赞、收藏、删除等操作后，快速显示结果提示，增强操作感知：

Code block

```
1  <template>
2    <button @click="collectGoods">收藏商品</button>
3  </template>
4
5  <script setup>
6    const collectGoods = async () => {
7      try {
8        // 执行收藏接口请求
9        await uni.request({
10          url: 'https://api.example.com/collect',
11          method: 'POST',
12          data: { goodsId: 123 }
13        })
14
15        // 收藏成功提示
16        uni.showToast({
17          title: '收藏成功',
18          icon: 'success',
19          duration: 1200 // 短时长，快速提示
20        })
21      } catch (err) {
22        uni.showToast({ title: '收藏失败', icon: 'error' })
23      }
24    }
25  </script>
```

五、常见问题与避坑指南

1. 提示文本显示不全？

原因：title 文本过长（超过 15 字），或部分平台对文本长度限制更严格；图标与文本排版冲突。

解决方案：

精简提示文本，核心信息控制在 10 字以内，避免换行。若文本必须较长，设置 `icon: 'none'`，取消图标释放空间。

2. 加载中提示无法关闭？

原因：使用 `icon: 'loading'` 时未手动调用 `uni.hideToast()`；hideToast 调用时机错误（如在请求完成前调用）。

解决方案：

确保在接口请求完成（成功/失败）后，务必调用 `uni.hideToast()`。避免在异步操作外提前调用 `hideToast`，可通过 `try/catch/finally` 确保调用：

Code block

```
1  const fetchData = async () => {
2    uni.showToast({ title: '加载中...', icon: 'loading', mask: true })
3    try {
4      const res = await uni.request({ url: 'xxx' })
5      // 处理数据
6    } catch (err) {
7      uni.showToast({ title: '失败', icon: 'error' })
8    } finally {
9      uni.hideToast() // 无论成功失败，都关闭加载提示
10   }
11 }
```

3. 自定义图标不显示？

原因：图标路径错误（使用网络路径或相对路径错误）；图标尺寸过大或格式不正确。

解决方案：

图标必须放在 `static` 目录下，使用绝对路径（如 `/static/icons/xxx.png`）。图标尺寸建议为 40x40px，格式为 PNG（避免使用 JPG 或 SVG）。检查图标文件是否存在，路径是否拼写错误（区分大小写）。

4. 多端显示样式不一致？

原因：各平台对 `showToast` 的默认样式做了适配，如 H5 端提示框更扁平，App 端更圆润。

解决方案：

核心功能（文本、图标）保持一致，接受轻微样式差异（符合多端用户习惯）。若需严格统一样式，可使用自定义组件（如基于 `uni-popup` 实现）替代 `showToast`。

5. 提示叠加显示？

原因：前一个提示未关闭时，再次调用 `showToast`，导致多个提示叠加。

解决方案：

调用 `showToast` 前，先调用 `uni.hideToast()` 关闭可能存在的旧提示。封装统一的提示工具函数，内置防叠加逻辑：

Code block

```
1  // 封装防叠加的 toast 工具函数
2  const showSafeToast = (options) => {
3    // 先关闭旧提示
```

```
4     uni.hideToast()
5     // 显示新提示
6     uni.showToast({
7         title: '默认提示',
8         ...options
9     })
10 }
11
12 // 使用
13 showSafeToast({ title: '操作成功', icon: 'success' })
```

六、总结

uni.showToast() 是 UniApp 中轻量级交互反馈的核心 API，使用时需把握以下核心原则：

1. **参数精简**：仅必选参数 `title` 即可实现基础提示，按需添加 `icon`、`duration` 等参数。
2. **场景匹配**：成功/失败用对应官方图标，加载中用 loading 图标并手动关闭，自定义图标需符合尺寸要求。
3. **体验优化**：重要操作加遮罩防穿透，提示文本清晰简洁，避免叠加显示。
4. **多端适配**：接受轻微样式差异，如需统一样式可使用自定义组件替代。

掌握 showToast 的用法，能有效提升用户操作感知，让交互更流畅自然。在实际开发中，建议封装统一的提示工具函数，简化调用并处理边界问题（如防叠加、统一样式）。

（注：文档部分内容可能由 AI 生成）